Zewail City of Science and Technology

Course Project (20% of Final Grade)
Team Size: 3-4 Students
Distributed Intelligent Campus IoT Environment

Objective: Develop an IoT platform that simulates a 200-room campus. The system optimizes energy autonomously, and remains operational during simulated network failures, all running within a self-hosted Dockerized environment.
The IoT system includes:

- Simulate hundreds of IoT devices
- Implement edge and cloud processing
- Maintain digital twins/ML processing
- Perform autonomous optimization
- Handle failures and network issues
- Provide secure APIs and dashboards
- Demonstrate scalability and resilience
- No physical hardware is required.

Project Phases Timeline

Project Phases	Deliverables	Grade	Deadline
Team Formation	Team size 3-4	**One grade penalty if not submitted on time**	Thursday 26/2/2026
Phase 1	Concurrent Simulation & Modeling	**5**	Sunday 18/3/2026
Phase 2	MQTT & CoAP Infrastructure & Processing	**6**	Wednesday 15/4/2026
Phase 3	Data Persistence (TDB)	**4.5**	Sunday TBD
Phase 4	Optimization, Resilience & Chaos Testing	**4.5**	Sunday TDB

This project is an opportunity to build a professional piece for your portfolio, something you can truly be proud to feature on your resume. Something to add to your GitHub repo when applying for jobs or internships. Your creativity is highly encouraged; feel free to put your own unique spin on the project's UI and Backend features within the bounds of the requirements.

Phase 1: Concurrent Simulation & Modeling (Details Added)

[Dr. Rakha added lots of details in this Phase to make it easier to understand and implement]

The focus of this phase is High-Concurrency Architectures, Deterministic Physics Logic, and Telemetry Generation. In this phase, each team will develop the "World Engine", the foundational data source for the entire IoT ecosystem. This engine is not a simple random number generator; it is a high-performance simulation capable of managing 200 concurrent room states in real-time.

1. The Virtual Campus "Fleet of Devices":

Each team must model a structural hierarchy consisting of 1 Building, 10 Floors, and 20 Rooms per floor, totaling 200 unique IoT nodes. When we talk about "modeling" in the context of a distributed system like this, we aren't talking about 3D models. We are talking about creating a Digital Blueprint in project code, defining how the system sees and organizes its data. Each room node must independently track and broadcast the following variables:

- **The Fleet Concept:** You treat the devices as a collective group. If you want to change the "Eco-Mode" temperature for the whole building, you send a "Fleet-wide Command." Keeping all 200 nodes (rooms) synchronized is important. If 190 rooms are reporting data but 10 are silent, we have a "Fleet Health" issue.
- **Sensors:**
  - Temperature (°C)
  - Humidity (%)
  - Occupancy (Boolean)
  - Ambient Light (Lux)
- **Actuators:**
  - HVAC Status (On/Off/Eco)
  - Smart Lighting Level (0-100%)

1.0. Fleet Health & Heartbeat Monitoring:

Each node must periodically publish a **heartbeat message**. If a node remains silent for > configurable timeout (e.g., 60 simulated seconds), the system must:

- Flag a Fleet Health Warning
- Log the anomaly in structured logs

1.1. The Data Structure Model:

Instead of having 200 separate variables, each team should create a **Class** or **Object** called Room. This class acts as a template. When the program starts, it "instantiates" (creates) 200 versions of this class.

What goes inside each Room object:

- ➢ **Identity:** building_id, floor_id, room_id.
- ➢ **State:** Current values for temp, humidity, occupancy, etc.
- ➢ **Logic:** A function that says, "If HVAC is ON, increase temp by 0.1."

1.2. State Persistence

The engine must:

- Persist room states to lightweight storage (using SQLite)
- Restore the previous state upon container restart
- Avoid a full reset on a crash

- In SQLite, they should maintain a table that represents the "Last Known Truth" for every room. Each row in the database should represent one room. When the engine "ticks," it updates these values. Here is the specific breakdown of what needs to be stored:

Column	Data Type	Why it must be stored
room_id	String (PK)	The unique key (e.g., b01-f05-r502) to look up the state.
last_temp	Float	To prevent the temp from resetting to a default (like 0°C or 20°C) on restart.
last_humidity	Float	To maintain the atmospheric continuity.
hvac_mode	String	If the HVAC was "ECO" before the crash, it must remain "ECO" after.
target_temp	Float	Stores the user's desired setpoint so it isn't lost.
last_update	Integer	The Unix timestamp of the last successful simulation tick.

- When the Python script starts, the very first thing it does is query the SQLite DB. If the DB is empty, Initialize rooms with default values (e.g., 22°C). If the DB has data, use those values to populate the Room objects. This ensures that if the container crashes at 2:00 PM, it reboots at 2:01 PM and the building "remembers" exactly where it was. You shouldn't write to SQLite every 5 seconds for all 200 rooms (this causes high Disk I/O). Implement a "Sync" every 30 or 60 seconds, or whenever a Command (like changing HVAC mode) is received. This creates a "Save Point."

1.3. The Hierarchy Model (Addressing):

Modeling the hierarchy is about how the system identifies a specific node. Each team must decide how to address a room. This is usually reflected in the MQTT Topic Structure. Teams must use a structured MQTT path: campus/bldg_01/floor_##/room_###/telemetry. Instead of a random ID like sensor_99, the model forces a structured path such as: campus/bldg_01/floor_05/room_502/telemetry.

1.4 Conceptual Python Implementation (Example):

```
import asyncio, time, aiosqlite

class Room:
    def __init__(self, b, f, r, state=None):
        self.id = f"b{b}-f{f}-r{r}"
        self.path = f"campus/b{b}/f{f}/r{r}"
        # Restore state from SQLite if available, else default
        self.temp = state['temp'] if state else 22.0
        self.hvac = state['hvac'] if state else "OFF"

    async def run_simulation(self, mqtt, db):
        await asyncio.sleep(random.uniform(0, 5)) # Startup Jitter
        while True:
            start = time.perf_counter()

            # 1. Physics & Persistence
            if self.hvac == "ON": self.temp += 0.1
            await db.execute("REPLACE INTO states VALUES (?,?,?)", (self.id, sel
            await db.commit()

            # 2. Telemetry Broadcast
            payload = {"id": self.id, "ts": int(time.time()), "temp": round(self
            await mqtt.publish(f"{self.path}/telemetry", payload)

            # 3. Drift Compensation
            await asyncio.sleep(max(0, 5 - (time.perf_counter() - start)))
```

1.5 Creating the "Fleet":

To "Model the Fleet," each team won't have to type 200 lines. You may use nested loops during the initialization phase of their engine:

```
rooms = []
for f in range(1, 11): # 10 Floors
    for r in range(1, 21): # 20 Rooms per floor
        new_room = Room(building="b01", floor=f, room_num=r)
        rooms.append(new_room)
```

1.6. Config-Driven Architecture:

All system parameters must be configurable via:

- YAML file OR
- Environment variables (such as .envOptions)

Including:

- Room count
- Thermal constants
- Publish frequency
- Fault probability
- Time acceleration factor

Scaling to 1000 rooms must require only a config change.

2. Deterministic Physics-Based Logic:

Teams must replace stochastic (random) data with a deterministic thermal model. Instead of temperatures "teleporting" from 20°C to 25°C, the teams must write a mathematical function that simulates how heat actually moves. Every room must behave as a physical entity where the temperature (T_room) reacts to its environment over time, Delta_t. Each team will implement a logic-based update loop:

- **Thermal Leakage:** The room temperature must naturally drift toward the "Outside Ambient Temperature" based on a thermal conductivity constant.
- **Actuator Impact:** When the HVAC actuator is set to "ON," the temperature must increase or decrease toward the setpoint at a realistic rate (e.g., 0.1°C per simulation tick).
- **Environmental Correlation:** If "Occupancy" is true, "Ambient Light" should stay above a certain threshold (simulating human activity), creating a realistic data correlation for later analytics.

For more details, each team should implement these three components within their Room class logic.

2.1. The Thermal Model Equation

Teams should use a simplified version of **Newton's Law of Cooling**. In every "tick" of the simulation (e.g., every 5 seconds == Delta_t), the room temperature is updated based on this formula:

$$T_{next} = T_{current} + [\alpha \cdot (T_{outside} - T_{current})] + [\beta \cdot \mathrm{HVAC}_{power}]$$

- **(Thermal Leakage "alpha":** This is the "Insulation Constant" (e.g., 0.01). If the room is 20°C and it's 40°C outside, the room will slowly warm up.
- **Actuator Impact "beta":** This is the "HVAC Strength" (e.g., 0.2).
    - If HVAC is ON (Heating): HVACpower = 1

- If HVAC is OFF: HVACpower = 0
- If HVAC is ECO: HVACpower = 0.5

Environmental Correlation Rules:

- If occupancy == true , then ambient light must exceed threshold
- Occupancy must increase the temperature slightly
- Night cycle must influence outside temperature (via virtual clock)

2.2. Fault Modeling:

Each team must implement at least 3 fault types:

1. Sensor Drift (gradual bias)
2. Frozen Sensor (stuck value)
3. Telemetry Delay
4. Node Dropout (temporary silence)

Fault rate must be configurable.

2.3. The Simulation "Tick" & Update Loop

The "Tick" is the asynchronous heartbeat of the engine. In the Python asyncio code, the "Modeling" functions like a state machine that runs continuously across 200 independent tasks.

Each room runs as an independent asyncio task. Per simulation tick, a node must:

1. **Read Current State:** Retrieve the current T_room, T_outside, and hvac_status.
2. **Apply Physics Update:** Calculate thermal leakage and actuator impact.
   - Example: leakage = 0.01 * (outside_temp - current_temp)
   - Example: change = 0.5 (if HVAC is ON) or 0.0 (if OFF).
   - Formula: T_next = T_curr + leakage + change
3. **Apply Logic & Faults:** Execute environmental correlations (e.g., Occupancy and Light) and inject faults if the "Chaos" probability is triggered.
4. **Validate State:** Ensure values remain within the standard ranges defined in the Data Specification Table.
5. **Broadcast Telemetry:** Publish the resulting JSON payload to the MQTT broker.
6. **Publish Heartbeat:** Send a separate "Healthy" status signal to the Fleet Monitoring topic.

Technical Constraints for Implementation:

1. Non-Blocking Execution & Event Loop Integrity

In an asynchronous environment, the "Event Loop" is a single-threaded manager. If one task "blocks" the thread, all 200 rooms stop moving.

- **<u>The Constraint:</u>** Teams must use await asyncio.sleep(). The use of time.sleep() or heavy synchronous I/O (like standard requests or pika) is strictly prohibited.

- **The Rationale:** Synchronous calls stop the loop. If Room #1 takes 100ms to process a "blocked" request, and there are 200 rooms, Room #200 will experience a 20-second delay before it even starts its tick.
- **Implementation Requirement:** All network calls (MQTT publishes) and file I/O must use non-blocking libraries (e.g., gmqtt).

2. Precision Drift Compensation

Every line of code takes time to execute. If a team simply calls sleep(5), the actual cycle time becomes 5 seconds + execution time. Over a 30-minute test, the simulation will "drift" and fall behind the real-world clock. To ensure the simulation doesn't "slip" over time, teams should subtract the physics calculation time from the sleep interval:

- **The Constraint:** Teams must calculate the delta of their execution time.
- **The Formula:**
    - **actual_sleep** = max(0, target_interval - processing_time)
- **The Rationale:** This ensures that even if the physics math or JSON serialization takes 200ms, the next tick still starts exactly 5 seconds after the previous one began. This is critical for keeping the Time-Series database aligned and ensuring that historical trends remain chronologically accurate during long-term analysis.

3. Startup Jitter & "Thundering Herd" Prevention

When a distributed system starts, having 200 nodes attempt to connect to a broker at the exact same millisecond creates a "Thundering Herd" effect, often crashing the broker or triggering rate-limits.

- **The Constraint:** Each room task must initialize with a randomized offset: await asyncio.sleep(random.uniform(0, max_jitter)).
- **The Rationale:** By spreading the 200 "ticks" across the 5-second window, the MQTT broker sees a smooth, consistent flow of 40 messages per second rather than a massive burst of 200 messages every 5 seconds.
- **The Result:** This levels out CPU and Network usage, ensuring the Event Loop Latency remains < 200ms as required.

In the Python asyncio code, the "Modeling" looks like a state machine that runs continuously.

Example Logic for a single "Tick":

1. Read current state: Get current T_room, T_outside, and hvac_status.
2. Calculate Leakage: leakage = 0.01 * (outside_temp - current_temp)
3. Calculate HVAC effect: * If status == "ON", change = 0.5

    - If status == "OFF", change = 0
4. Update State: T_next = current_temp + leakage + change
5. Later, broadcast: Send this new_temp via MQTT.

Each room runs as an independent asyncio task:

Per simulation tick:

1. Read the current state
2. Apply physics update
3. Apply the actuator impact
4. Apply faults (if active)
5. Validate state ranges
6. Publish telemetry (if interval reached)
7. Publish heartbeat

The event loop must remain non-blocking.

3. Wokwi Hardware Prototyping (Proof of Concept)

Before scaling to the full engine, each team will design a "Reference Room" using the Wokwi simulator. The Wokwi POC acts as the **blueprint**. Once one room is working in Wokwi, your team can take the logic (the math and the JSON structure), port it to the Python asyncio engine and loop to "clone" that logic 200 times.

- **Logic Validation:** Teams will write MicroPython code for an ESP32 microcontroller within Wokwi to interface with virtual DHT22 and PIR sensors. In addition to Simulating Timestamp (NTP & Virtual WiFi as Microcontrollers do not know what time it is). The MicroPython code that handles a sensor read (e.g., waiting for a DHT22 to stabilize) and logic.
- **Protocol Handshake:** The Wokwi POC must successfully connect to an MQTT broker and publish a valid JSON payload.
- **Porting:** It is much easier to debug **one** virtual device than **200** simultaneous threads. Once the hardware logic and communication schema are validated in Wokwi, each team will port this logic into the high-concurrency Python engine to handle the remaining 199 nodes. As an example, the logic could be "Read the sensor every 1 second, but only publish to MQTT every 5 seconds OR if the temperature changes by more than 0.5°C.".
- JSON Schema Validation, the MicroPython firmware must: Validate outgoing telemetry structure, Reject malformed command payloads, Log invalid message attempts.

4. High-Concurrency Engine (Python asyncio)

To simulate 200 nodes within a single container without performance degradation, each team must utilize asynchronous programming.

- **Task Management:** Every room must be managed as a separate asyncio task.

- **Non-Blocking I/O:** The engine must be able to calculate physics updates for all 200 rooms and push their data to the MQTT broker simultaneously without the event loop "lagging" or blocking.
- **Scalability:** The engine should be designed such that increasing the room count from 200 to 1,000 would only require configuration changes, not a rewrite of the core logic.

5. Data Payload Schema

Teams must define and implement a standardized JSON telemetry format that includes metadata for routing. Example:

5.1 Telemetry Schema:

For the Telemetry Message (Sensor Data), the frequency of the telemetry data is configurable, and the ultimate direction of the data is to move from the simulated device to the MQTT Broker, then to the Database/Cloud. Each team must ensure that the "World Engine" publishes a flat JSON object representing the current state of a room, following just an example:

```
{
  "metadata": {
    "sensor_id": "b01-f05-r202",
    "building": "b01",
    "floor": 5,
    "room": 202,
    "timestamp": 1708435200
  },
  "sensors": {
    "temperature": 22.5,
    "humidity": 45.2,
    "occupancy": true,
    "light_level": 450
  },
  "actuators": {
    "hvac_mode": "eco",
    "lighting_dimmer": 75
  }
}
```

5.2. The Command Message (Actuator Control):

The direction of such messages will be from Backend/Dashboard to the MQTT Broker, then to Simulator (Device). Below is the suggested (just as an example) JSON Schema:

```
{
  "command_id": "cmd-99812",
  "target_device": "b01-f05-r202",
  "action": "set_hvac",
  "value": "ON",
  "timestamp": 1708435300
}
```

5.3 Data Specification Table:
Each team must validate its physics model outputs against these standard ranges and types to maintain system realism.

Field	Data Type	Unit / Range	Description
sensor_id	String	Unique Slug	Format: [Bld]-[Flr]-[Room] (e.g., b01-f10-r1001)
timestamp	Integer	Unix Epoch	Seconds since Jan 1, 1970.
temperature	Float	15.0 to 50.0	Measured in Celsius (°C).
humidity	Float	0.0 to 100.0	Relative Humidity percentage.
occupancy	Boolean	true/false	Presence detection in the room.
light_level	Integer	0 to 1000	Measured in Lux.
hvac_mode	String	ON, OFF, ECO	The current state of climate control.
lighting_dimmer	Integer	0 to 100	Percentage of brightness.

6. Performance Testing

Teams must demonstrate benchmark testing as follows:

- 200 rooms
- 5-second publish interval
- Event loop latency < 200 ms
- Continuous operation for 30 minutes
- CPU and memory usage reporting

7. Phase 1 Deliverables

Each team must submit:

- **Everything discussed above. Plus the following:**
- **The World Engine:** A fully functional Python application using asyncio.
- **Dockerization:** A Dockerfile that packages the engine and its dependencies.
- **POC Documentation:** A short report (i.e., less than 10 pages) (including screenshots) and a short video demonstrating the Wokwi "Reference Room" successfully publishing data.
- **Use Github repository to track all your project code and progress.**
- **Live Stream:** The container must be seen broadcasting continuous, logically consistent data to a local MQTT broker.

Phase 2: MQTT & CoAP Infrastructure & Processing

To implement a high-performance, hybrid Phase 2 environment, the technical stack must be chosen for its ability to handle asynchronous I/O and multi-protocol concurrency. The goal is to move away from simple scripts toward a robust, enterprise-grade architecture. This phase focuses on the transition from data generation to Network Orchestration. The goal is a professional, distributed environment in which MQTT and CoAP operate as native, high-performance protocols within a HiveMQ-centered, Thingsboard backbone.

---

1. The Hybrid World Engine (Python asyncio)

- **100 MQTT Nodes:** Implementation of 100 asynchronous tasks using the gmqtt library. Each node must maintain a persistent TCP connection to HiveMQ, featuring a unique ClientID and a Last Will and Testament (LWT).
- **100 CoAP Nodes:** Implementation of 100 asynchronous CoAP servers using aiocoap. These nodes must host Observable Resources (RFC 7641: Observe extension that turns a one-time request into a long-term subscription, without it, gateways will cause polling issues.), allowing the Edge Gateway to receive telemetry updates via UDP only when values change. aiocoap has built-in, native support for RFC 7641
- **Physics-Transport Integration:** The physics logic from Phase 1 must be seamlessly piped into these two protocol stacks, ensuring that the "Thermal Model" data is transmitted accurately regardless of the transport layer.

2. Distributed Brokerage & Gateway Infrastructure (Docker)

1. We have 10 Gateways (one for each floor). Each Gateway is responsible for 10 CoAP Nodes (Rooms 10 on its floor). Similarly, each Gateway is also responsible for 10 MQTT Nodes on that same floor. As a total workload per a Gateway, we have 20 Rooms (10 CoAP + 10

MQTT). These gateways are the reflexes. They sit on the same local network as the rooms. They "watch" the 10 MQTT & 10 CoAP rooms on their floor and immediately report any changes up to the "Brain" (HiveMQ/ThingsBoard).

2. **HiveMQ Backbone is** A Dockerized HiveMQ Community Edition serving as the central **"Campus Hub"** for local MQTT traffic between nodes, gateways, and edge logic.
3. **ThingsBoard Cloud Platform (Local On-premises Simulation):** A Dockerized ThingsBoard Community Edition (or ThingsBoard PE) serving as the on-premises cloud-layer platform.
4. **MQTT Integration** The system implements a "Remote Observer" model, publishing raw MQTT to HiveMQ and having ThingsBoard "subscribe" to it via an MQTT Integration. This ensures that the local campus network remains functional even if the connection to the ThingsBoard container is throttled or interrupted.
5. **The CoAP-to-Cloud Pipeline:** Here is how the data flows from a CoAP room to the ThingsBoard dashboard:
    1. **The Observation (Edge):** Each of the 10 Gateways acts as a CoAP Client. It "Observes" (RFC 7641) the 10 rooms on its floor.
    2. **The Trigger:** When a room's temperature changes, the CoAP node pushes a UDP packet to the Gateway.
    3. **The Translation:** Node-RED receives the CoAP payload and immediately **re-packages** it into an MQTT message.
    4. **The Bridge:** The Gateway publishes that MQTT message to HiveMQ using the same topic structure as the native MQTT nodes (e.g., campus/b01/f01/r201/telemetry).
    5. **The Unified Stream:** ThingsBoard—which is already "listening" to HiveMQ—receives the message. It doesn't even know it started as a CoAP packet!

<u>ThingsBoard is responsible for:</u>

- **Device Registry & Provisioning:** All 200 nodes (100 MQTT + 100 CoAP) must be registered as ThingsBoard Devices, organized under a Device Profile per node type (e.g., MQTT-ThermalSensor).
- **Asset Hierarchy:** Campus → Building → Floor → Room asset structure mirroring the physical topology, with devices linked to their corresponding Room assets via ThingsBoard **Relations**.
- **Rule Engine Processing:** ThingsBoard Rule Chains replace cloud-side logic previously handled at the edge, including alarm generation, threshold-based triggers, and telemetry routing.
- **Real-Time Dashboards:** All monitoring, fleet health, and latency visualization is handled through ThingsBoard Dashboards (replacing standalone Node-RED UIs for the cloud layer).

- **HiveMQ Edge / Thingsboard Node-RED Gateways:** Deployment of 10 Floor-level containers (Gateways). These must be configured to ingest data from both the MQTT and CoAP segments simultaneously.
- **Unified Topic & URI Hierarchy:**
  - **MQTT:** campus/b01/f##/r###/telemetry
  - **CoAP:** coap://[node_ip]/f##/r###/telemetry
- **The Command Pipeline:** the data flow is not only telemetry (upstream) that goes from the Node to the Cloud. But how does a Command (downstream) go from the ThingsBoard Dashboard back to a room actuator. We have two flows described below:

<u>Flow 1: MQTT Downstream (The "Listener" Model)</u>

Since MQTT is a persistent TCP connection, the node is always "on the line" waiting for a shout from the broker.

1. **Trigger:** An operator clicks a "Toggle Heat" button on the ThingsBoard Dashboard.
2. **Cloud Publication:** ThingsBoard publishes an MQTT message to the HiveMQ Backbone on a specific command topic: campus/b01/f01/r101/cmd.
3. **Broker Routing:** HiveMQ identifies that the Room 101 Node has an active subscription to that topic and pushes the packet down.
4. **Reliability (QoS 2):** Because this is a command (an "Event"), it is sent with Quality of Service 2 to ensure it is delivered exactly once.
5. **Node Reception:** The gmqtt library in the Python script triggers the on_message callback.
6. **Virtual Actuation:** The script updates its internal physics variable: self.hvac_active = True.

<u>Flow 2: CoAP Downstream (The "Proxy" Model)</u>

Since CoAP is UDP-based and "stateless," the node doesn't stay connected. The Gateway must act as a translator to deliver the message.

1. **Trigger:** An operator clicks "Toggle Heat" for a CoAP-enabled room on the **ThingsBoard Dashboard**.
2. **Cloud Publication:** ThingsBoard publishes the command to HiveMQ on the topic: campus/b01/f01/r201/cmd.
3. **Gateway Ingestion:** The Floor 01 Gateway (Node-RED) is subscribed to all floor commands. it intercepts this MQTT message.
4. **Protocol Translation:** Node-RED maps the MQTT topic to the specific Local IP Address of the Room 201 CoAP server.
5. **UDP Transmission:** The Gateway sends a CoAP PUT request to the node's resource: coap://[node_ip]/f01/r201/actuators/hvac.
6. **Reliability (CON):** The message is sent as Confirmable (CON). The Room Node must send a UDP "ACK" back to the Gateway to confirm the "Muscle" received the signal.

The Gateway should then publish an MQTT message back to ThingsBoard (e.g., .../response) to confirm the command was successful.

7. **Virtual Actuation:** The aiocoap server in the Python script receives the PUT and updates: self.hvac_active = True.

3. Reliability & Deduplication Audit

- **Command Integrity (QoS 2):** Implementation of MQTT QoS 2 (Exactly Once) for all critical actuator commands (e.g., "Emergency Lockout") to prevent race conditions.
- **Alert Reliability (CON):** Use of Confirmable (CON) messages for CoAP-based "Sentinel" alerts (e.g., Smoke or Intruder detection).
- **The DUP Flag Handler:** A documented logic flow showing how the system identifies the DUP (Duplicate) flag in incoming packets. The backend must demonstrate that retransmitted messages (due to network lag) are identified and not processed as new events.

4. Edge Intelligence & Data Thinning (Node-RED)

- **Dual-Protocol Orchestration:** Exported Node-RED flows showing the concurrent ingestion of MQTT subscriptions and CoAP "Observations."
- **Data Aggregation:** Implementation of "Edge Thinning", where the Gateway calculates a 60-second floor average and publishes a single "State Summary" to the HiveMQ Cloud to reduce network congestion.
- **Offline Autonomy:** A demonstration that if the "Cloud" connection is lost, the Edge Gateway continues to process local room logic (e.g., turning off lights if occupancy is false) using local MQTT/CoAP traffic.

5. Security & Identity Framework

- **Dual-Layer Encryption:** Implementation of **TLS** for all MQTT connections and **DTLS (Datagram Transport Layer Security)** for CoAP traffic.
- **HiveMQ ACLs:** A configuration file for HiveMQ **Access Control Lists** ensuring that a node on "Floor 01" cannot publish to or spoof a node on "Floor 10."
- **Authentication:** Proof of unique username/password or Pre-Shared Key (PSK) authentication for every simulated node.

6. Performance & Monitoring Report

The "HiveMQ Pulse" Snapshot: Visual proof from the HiveMQ Control Center showing 100 active MQTT clients and a stable "Messages Per Second" metric.

The "Unified Fleet Health" Dashboard: A ThingsBoard UI showing a real-time status grid for all 200 rooms.

- **MQTT Nodes:** Must use LWT (Last Will and Testament) to trigger an "Offline" status if the connection drops.
- **CoAP Nodes:** Must use a Timeout/Heartbeat logic (if no update is received within "X" seconds, the room is marked "Dead").

The "Round-Trip" Latency Benchmark: A data-backed report measuring the time from a ThingsBoard Command (Southbound) to the Python Node's State Change (Virtual Actuator), and finally the Telemetry Update (Northbound) appearing back on the dashboard.

- **Target:** Total Round-Trip Time (RTT) must be < 500ms under a 100% simulation load.

Packet Integrity Audit: A brief log analysis proving that QoS 2 and CoAP CON messages were used for all "Alert" and "Command" events, with zero packet loss recorded during a 5-minute stress test.

Technical Summary for Submission

Component	Technology	Implementation Detail
Concurrency	asyncio	Managing 100+ tasks/servers.
Backbone	HiveMQ CE	Central orchestration & Dashboard.
MQTT Client	gmqtt	Async, persistent TCP.
CoAP Server	aiocoap	Resource-based, event-driven UDP.
Edge Logic	Node-Red Gateway	Data aggregation/thinning & Protocol unification, bi-directional protocol translation.
Cloud Integration	Local ThingsBoard as an on-premises simulation	Asset Hierarchies, Rule Engines, and Command UI
Security	TLS / DTLS	Identity & Privacy across all 200 nodes.

Reliability	QoS 2 / CON / DUP	Zero-loss, zero-duplicate message logic.

References:

6. https://thingsboard.io/
7. https://thingsboard.io/docs/
8. https://thingsboard.io/docs/user-guide/device-credentials/
9. https://thingsboard.io/docs/reference/mqtt-api/

Phase 2 Deliverables

All Details mentioned above must be satisfied. Here is a summary of the deliverables list:

1. Continue on Phase1, the "World Engine" Source Code (Python)

- **Asynchronous Node Script:** A single, optimized Python script (or package) using asyncio that can spawn 100 MQTT clients (gmqtt) and 100 CoAP servers (aiocoap) simultaneously.
- **Thermal Logic Integration:** The code must show the Phase 1 physics engine feeding data into both protocol stacks.
- **Virtual Actuator Logic:** Implementation of internal state variables (e.g., self.hvac_status) that change based on incoming MQTT commands or CoAP PUT requests.

2. The "Gateway" Orchestration (Node-RED)

- **Flow Exports (.json):** The JSON files for the 10 Floor Gateways.
- **Protocol Translation Logic:** Visible "nodes" in the flow that show:
  - MQTT Subscription to Internal Logic.
  - CoAP Observation to MQTT Re-packaging.
  - Southbound Command Mapping (MQTT to CoAP PUT).
- **Edge Thinning:** Logic showing the 60-second mathematical averaging of floor data before cloud transmission.

3. The "Mind" Configuration (ThingsBoard)

- **Device & Asset Export:** A CSV or JSON export of the 200-device registry and the Campus to Building to Floor to Room hierarchy.
- **Rule Chains:** A screenshot or export of the Rule Engine logic used to route telemetry and trigger basic threshold alarms.
- **NOC Dashboard:** A functional ThingsBoard dashboard showing real-time telemetry from both MQTT and CoAP rooms, including "Online/Offline" status.

4. Infrastructure & Security (Docker & Configs)

- **Docker Compose File:** A docker-compose.yml that launches HiveMQ, ThingsBoard, and the 10 Node-RED containers in a unified network.
- **HiveMQ ACL File:** The .access or configuration file proving that nodes are restricted to their specific floor topics.
- **Security Certificates:** Proof of generated self-signed certificates or PSK (Pre-Shared Keys) used for TLS/DTLS encryption.

5. The Performance & Reliability Report (PDF ~ 5 Pages)

- **The "Stress Test" Evidence:** Screenshots of the HiveMQ Control Center showing 100+ active clients and stable throughput.
- **The Latency Audit:** A table or chart showing the Round-Trip Time (RTT) for commands (from Dashboard click to Node acknowledgment) staying under 500ms.
- **The Reliability Proof:** A short technical write-up (with logs) demonstrating how the DUP flag was handled and how QoS 2 / CON ensured message delivery during a simulated network flicker.
- Any details mentioned in the phase description. Add more pages as you want to explain or show your hard work and results.

Phase 3: Data Persistence (TDB)

[Dr. Rakha may apply changes, add more details, and requirements to this Phase later as more course materials are covered.]

Phase 4: Optimization, Resilience & Chaos Testing

[Dr. Rakha may apply changes, add more details, and requirements to this Phase later as more course materials are covered.]

Bonus:

Dr. Rakha may or may not add a bonus to this project, yet to be decided.

Important Rules:

1. For each deliverable, **if a late submission is permitted, a 25% penalty will be incurred** for each day after the deadline. After two overdue days, no submissions will be accepted.
2. Each deliverable must be uploaded to the **Google Classroom** assigned assignment and pushed to the repository **before the submission deadline**.
3. Teams are encouraged to be from the same lab. Team members can be from different LABS, but NOTE that: It will be your FULL RESPONSIBILITY to attend the evaluation session. Any student who misses the scheduled evaluation session will automatically receive ZERO.
4. The deliverable version uploaded to the **Google Classroom** assignment is the one that will be evaluated. If it's a team project, the team representative can just upload the deliverable for other members. The GitHub version will serve only as a reference to track progress.
5. **Version Control:** All projects must use GitHub to track progress. Commits should reflect contributions from all members. **GitHub is mandatory in the evaluation process** and must follow the "GitHub Repository Structure" mentioned. Failing to maintain an updated GitHub repository for your projects may result in a penalty of **up to 20%** for the deliverable under evaluation.
6. **Documentation Requirements:** Each deliverable must include clear and complete documentation as outlined in the tasks, if any.
7. **Although this is a team project, every member is expected to fully understand all submitted work. TAs may ask any student about any part of the deliverable** (diagrams, code, documentation, etc). Any student unable to explain their project/code may lose marks individually, even if the deliverable was submitted.
8. **Cheating** could result in a negative grade (up to - 10 Course Grades).
9. **Academic Integrity Reminder**: Directly copying or pasting content from AI tools (such as ChatGPT or others) into your course deliverables is considered academic misconduct (Up to cheating). Your submitted work should reflect your own effort, writing style, creativity, and uniqueness. You may use AI tools to support your learning or improve your understanding, but the final content must be genuinely your own. All submissions/deliverables are subject to plagiarism and AI-detection checks. If there is reasonable suspicion of AI-generated or copied content, the lecturer/TA reserves the right to:
    - Assign a penalty for **cheating** if misuse is confirmed.
    - Request additional work or conduct an oral examination to verify authorship and understanding of the submitted material.
10. Each deliverable submission (when needed) must include **clear documentation** as required.

GitHub Repository & Version Control Requirement

Each team must create and maintain a dedicated GitHub repository for this project. All development work must be committed regularly throughout the project period. The repository will be used to track progress, evaluate development consistency, and verify individual contributions.

Teams are expected to make meaningful, incremental commits that reflect continuous development rather than uploading the entire project at once. Commit messages should be clear and descriptive. The repository must include a well-organized folder structure, source code, assets (where applicable), and a README file explaining how to run the project.

Failure to demonstrate a consistent development history through GitHub commit activity may impact the project evaluation.

Grading Policy

- Understanding deliverables and their details will be evaluated individually for each team member.
- Your project GitHub is part of the evaluation weight.
- All the mentioned requirements in each deliverable are part of the evaluation criteria weights.

Deliverable Discussion/Response Policy:

- If a student is asked a question during evaluation and does not answer, **5 points will be deducted**.
- If the student fails to answer a second question, **an additional 5 points will be deducted**.
- If the student fails to answer a third question, **the student will receive 0 for the Individual Understanding mark**.
- If the student fails to answer a fourth question, the student will receive 0 for the deliverable itself, regardless of the team's mark

Project Deliverable Documents

All project deliverables should have the following cover page:
Zewail City of Science, Technology and Innovation
University of Science and Technology
School of Computational Sciences and Artificial Intelligence

SWAPD453- Spring 2026

IoT App Devs
<Project Name>

Student#1 <Name> <ID>
Student#2 <Name> <ID>

<Date>